

LAB GUIDE WITH SOLUTIONS



Module 2: SQL Fundamentals (DDL & DML)

Module 2 - 8 Hours

SQL Fundamentals (DDL & DML)

How to use this lab guide

Eight labs, sized for a four-hour session. Solutions and expected output are inline. The unguided practice is in [M02_Exercises.pdf](#).

Lab	Title	Time
2.1	Prove the data type rules	25 min
2.2	Create the Smart Saver table	30 min
2.3	Break every constraint on purpose	30 min
2.4	ALTER the table as requirements change	25 min
2.5	Load the pilot bookings	30 min
2.6	The maturity run: safe UPDATE and DELETE	30 min
2.7	Filtering, NULL and sorting	35 min
2.8	DELETE, TRUNCATE and DROP compared	15 min

Start position: connected to [icicibank](#) with the nine tables from Module 1 loaded. Check with `\dt` before you begin. Keep a file called `module2_work.sql` and paste everything into it.

Lab 2.1 - Prove the data type rules

Objective: convince yourself, with output, why money is `NUMERIC` and not `FLOAT`.

Step 1 - Floating point arithmetic

```
SELECT CAST(0.1 AS REAL) + CAST(0.2 AS REAL)      AS float_sum,
       CAST(0.1 AS REAL) + CAST(0.2 AS REAL) = 0.3 AS float_equals_point3;
```

```
float_sum      | float_equals_point3
-----+-----
0.3000000119209290 | f
```

```
SELECT CAST(0.1 AS NUMERIC) + CAST(0.2 AS NUMERIC)      AS exact_sum,
       CAST(0.1 AS NUMERIC) + CAST(0.2 AS NUMERIC) = 0.3 AS exact_equals_point3;
```

```
exact_sum | exact_equals_point3
-----+-----
0.3 | t
```

Step 2 - Accumulate the error the way a bank would

```
CREATE TABLE type_demo (
    txn_no      INTEGER,
    amount_float REAL,
    amount_exact NUMERIC(15,2)
);

INSERT INTO type_demo
SELECT g, 0.10, 0.10 FROM generate_series(1, 100000) g;

SELECT SUM(amount_float) AS float_total,
       SUM(amount_exact) AS exact_total
FROM   type_demo;
```

```
float_total | exact_total
-----+-----
10000.0007629395 | 10000.00
```

One hundred thousand ten-paisa credits should total exactly 10,000 rupees. The floating point column is out by more than seven paisa. ICICI Bank posts two million transactions a month.

Step 3 - CHAR padding

```
SELECT LENGTH(CAST('INR' AS CHAR(10))) AS char_length,
       LENGTH(CAST('INR' AS VARCHAR(10))) AS varchar_length,
       '[' || CAST('INR' AS CHAR(10)) || ']' AS padded;
```

```
char_length | varchar_length | padded
-----+-----+-----
10 | 3 | [INR ]
```

CHAR pads with spaces to the declared length. That is fine for `currency CHAR(3)` where every value really is three characters, and it is a bug waiting to happen on a name column.

Step 4 - Clean up

```
DROP TABLE type_demo;
```

What just happened

You produced the evidence for the rule. When a developer proposes `FLOAT` for an amount column, this is the query to run in front of them.

Lab 2.2 - Create the Smart Saver table

Objective: build the table that ticket ICICI-4482 asked for, with every rule declared.

Step 1 - Create it

```
CREATE TABLE fixed_deposit (  
    deposit_id      INTEGER      NOT NULL,  
    customer_id     INTEGER      NOT NULL,  
    branch_id       INTEGER      NOT NULL,  
    receipt_number   VARCHAR(20)  NOT NULL,  
    principal_amount NUMERIC(15,2) NOT NULL,  
    interest_rate    NUMERIC(5,2) NOT NULL,  
    tenure_days      SMALLINT     NOT NULL,  
    booked_date      DATE          DEFAULT CURRENT_DATE NOT NULL,  
    maturity_date    DATE          NOT NULL,  
    auto_renew       CHAR(1)       DEFAULT 'N' NOT NULL,  
    status           VARCHAR(12)   DEFAULT 'ACTIVE' NOT NULL,  
    CONSTRAINT pk_fixed_deposit PRIMARY KEY (deposit_id),  
    CONSTRAINT uq_fd_receipt  UNIQUE (receipt_number),  
    CONSTRAINT fk_fd_customer FOREIGN KEY (customer_id) REFERENCES customer(customer_id),  
    CONSTRAINT fk_fd_branch   FOREIGN KEY (branch_id)   REFERENCES branch(branch_id),  
    CONSTRAINT ck_fd_principal CHECK (principal_amount >= 10000),  
    CONSTRAINT ck_fd_tenure    CHECK (tenure_days BETWEEN 7 AND 3650),  
    CONSTRAINT ck_fd_rate      CHECK (interest_rate > 0 AND interest_rate <= 15),  
    CONSTRAINT ck_fd_renew     CHECK (auto_renew IN ('Y','N')),  
    CONSTRAINT ck_fd_status    CHECK (status IN ('ACTIVE','MATURED','BROKEN','CLOSED')),  
    CONSTRAINT ck_fd_maturity  CHECK (maturity_date > booked_date)  
);
```

Expected: `CREATE TABLE`.

Step 2 - Inspect what you built

```
\d fixed_deposit
```

Read the output carefully. It shows every column with its type and default, then `Indexes`: listing the primary key and unique index, then `Check constraints`: and `Foreign-key constraints`:. Everything you declared is now visible to anyone who inspects the table. **The schema documents itself.**

Step 3 - Confirm from the catalog

```
SELECT constraint_name, constraint_type  
FROM   information_schema.table_constraints  
WHERE  table_name = 'fixed_deposit'  
ORDER  BY constraint_type, constraint_name;
```

Expected: 1 primary key, 1 unique, 2 foreign keys and 6 check constraints, plus the automatically generated `NOT NULL` checks.

What just happened

Eleven columns and ten named rules. Notice that you have written no application code, and the Smart Saver product's business rules are already enforced for every application that will ever connect - the mobile app, the branch system, a batch job, and a DBA running a manual correction.

Lab 2.3 - Break every constraint on purpose

Objective: see each rejection message, so you recognise it in a log at 2 a.m.

Run each statement. **Record the error.** None of these should succeed.

1 - NOT NULL

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
 interest_rate, tenure_days, maturity_date)
VALUES (9001, 1001, 1, 'FDTEST01', NULL, 7.25, 365, DATE '2027-03-01');
```

```
ERROR: null value in column "principal_amount" of relation "fixed_deposit"
violates not-null constraint
```

2 - PRIMARY KEY duplicate

Insert one good row first:

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
 interest_rate, tenure_days, booked_date, maturity_date, auto_renew, status)
VALUES (9001, 1001, 1, 'FDTEST01', 50000, 7.25, 365,
 DATE '2026-03-01', DATE '2027-03-01', 'Y', 'ACTIVE');
```

Then repeat it with a different receipt number:

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
 interest_rate, tenure_days, booked_date, maturity_date, auto_renew, status)
VALUES (9001, 1002, 1, 'FDTEST02', 50000, 7.25, 365,
 DATE '2026-03-01', DATE '2027-03-01', 'Y', 'ACTIVE');
```

```
ERROR: duplicate key value violates unique constraint "pk_fixed_deposit"
DETAIL: Key (deposit_id)=(9001) already exists.
```

3 - UNIQUE duplicate

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
 interest_rate, tenure_days, booked_date, maturity_date, auto_renew, status)
VALUES (9002, 1002, 1, 'FDTEST01', 50000, 7.25, 365,
 DATE '2026-03-01', DATE '2027-03-01', 'Y', 'ACTIVE');
```

```
ERROR: duplicate key value violates unique constraint "uq_fd_receipt"
DETAIL: Key (receipt_number)=(FDTEST01) already exists.
```

Two customers cannot be given the same receipt number. In a branch, that is a fraud control.

4 - FOREIGN KEY, no such customer

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
 interest_rate, tenure_days, booked_date, maturity_date, auto_renew, status)
VALUES (9003, 7777, 1, 'FDTEST03', 50000, 7.25, 365,
       DATE '2026-03-01', DATE '2027-03-01', 'Y', 'ACTIVE');
```

```
ERROR: insert or update on table "fixed_deposit" violates foreign key
       constraint "fk_fd_customer"
DETAIL: Key (customer_id)=(7777) is not present in table "customer".
```

5 - CHECK, below the product minimum

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
 interest_rate, tenure_days, booked_date, maturity_date, auto_renew, status)
VALUES (9004, 1003, 1, 'FDTEST04', 5000, 7.25, 365,
       DATE '2026-03-01', DATE '2027-03-01', 'Y', 'ACTIVE');
```

```
ERROR: new row for relation "fixed_deposit" violates check constraint "ck_fd_principal"
```

This is why constraints are named. The message names the product rule, not a random code.

6 - CHECK across two columns

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
 interest_rate, tenure_days, booked_date, maturity_date, auto_renew, status)
VALUES (9005, 1003, 1, 'FDTEST05', 50000, 7.25, 365,
       DATE '2026-03-01', DATE '2025-03-01', 'Y', 'ACTIVE');
```

```
ERROR: new row for relation "fixed_deposit" violates check constraint "ck_fd_maturity"
```

A deposit cannot mature before it was booked.

7 - CHECK, invalid domain value

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
 interest_rate, tenure_days, booked_date, maturity_date, auto_renew, status)
VALUES (9006, 1003, 1, 'FDTEST06', 50000, 7.25, 365,
       DATE '2026-03-01', DATE '2027-03-01', 'MAYBE', 'ACTIVE');
```

```
ERROR: value too long for type character(1)
```

Interesting: the **type** rejected it before the check constraint got the chance. Try 'X' instead and you get `ck_fd_renew`. Types and constraints are two layers of the same defence.

8 - DEFAULT working

```
INSERT INTO fixed_deposit
  (deposit_id, customer_id, branch_id, receipt_number, principal_amount,
   interest_rate, tenure_days, maturity_date)
VALUES (9007, 1003, 1, 'FDTEST07', 50000, 7.25, 365, DATE '2027-03-01');

SELECT deposit_id, booked_date, auto_renew, status
FROM   fixed_deposit WHERE deposit_id = 9007;
```

deposit_id	booked_date	auto_renew	status
9007	2026-09-10	N	ACTIVE

Three columns were never mentioned in the INSERT, and all three have sensible values. `booked_date` shows today's date.

Clean up

```
DELETE FROM fixed_deposit WHERE deposit_id >= 9000;
SELECT COUNT(*) FROM fixed_deposit;    -- 0
```

What just happened

Seven attempts to put bad banking data into the database, seven refusals, and one demonstration of defaults. Every rejection came from the engine, with a message naming the rule. This is the C in ACID, made concrete.

Lab 2.4 - ALTER the table as requirements change

Objective: evolve a live table without losing data.

Step 1 - Product adds nomination

```
ALTER TABLE fixed_deposit ADD COLUMN nominee_name VARCHAR(60);
\d fixed_deposit
```

The column is nullable, which is correct - nomination is optional.

Step 2 - Try to add a mandatory column to a populated table

First put a row in:

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
 interest_rate, tenure_days, booked_date, maturity_date, auto_renew, status)
VALUES (9010, 1001, 1, 'FDTEST10', 50000, 7.25, 365,
       DATE '2026-03-01', DATE '2027-03-01', 'Y', 'ACTIVE');
```

Now:

```
ALTER TABLE fixed_deposit ADD COLUMN product_code VARCHAR(10) NOT NULL;
```

```
ERROR: column "product_code" of relation "fixed_deposit" contains null values
```

The engine cannot invent a value for the existing row. Supply one:

```
ALTER TABLE fixed_deposit
ADD COLUMN product_code VARCHAR(10) NOT NULL DEFAULT 'SMARTSAVE';

SELECT deposit_id, product_code FROM fixed_deposit WHERE deposit_id = 9010;
```

```
deposit_id | product_code
-----+-----
      9010 | SMARTSAVE
```

Step 3 - Widen a type for the NRI variant

```
ALTER TABLE fixed_deposit ALTER COLUMN interest_rate TYPE NUMERIC(6,3);
```

Widening is safe. **Narrowing is not** - try it and see:

```
ALTER TABLE fixed_deposit ALTER COLUMN receipt_number TYPE VARCHAR(5);
```

```
ERROR: value too long for type character varying(5)
```

The engine refuses rather than truncate a receipt number. Remember this in Module 11: a migration that maps a wide source column to a narrow target column either fails loudly or, on a less careful tool, silently truncates.

Step 4 - Change a business rule

The minimum booking rises to 25,000 from April.

```
ALTER TABLE fixed_deposit DROP CONSTRAINT ck_fd_principal;  
ALTER TABLE fixed_deposit  
  ADD CONSTRAINT ck_fd_principal CHECK (principal_amount >= 25000);
```

This succeeds because the single existing row has 50,000. If any row held less, the `ADD CONSTRAINT` would fail and tell you so. **Adding a constraint validates the data you already have** - a free audit.

Step 5 - Remove the pilot column

```
ALTER TABLE fixed_deposit DROP COLUMN product_code;  
DELETE FROM fixed_deposit WHERE deposit_id = 9010;
```

What just happened

You added a column, learned why a mandatory column needs a default, widened a type, tightened a business rule, and dropped a column, all on a table with data in it. This is what maintaining a banking schema actually looks like.

Lab 2.5 - Load the pilot bookings

Objective: get the twenty branch pilot bookings in, three different ways.

Step 1 - One row, columns named

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
interest_rate, tenure_days, booked_date, maturity_date, auto_renew,
nominee_name, status)
VALUES
(7001, 1001, 1, 'FD2026000001', 25000.00, 6.75, 90,
DATE '2026-03-01', DATE '2026-05-30', 'N', 'Priya Verma', 'ACTIVE');
```

Step 2 - Several rows in one statement

```
INSERT INTO fixed_deposit
(deposit_id, customer_id, branch_id, receipt_number, principal_amount,
interest_rate, tenure_days, booked_date, maturity_date, auto_renew,
nominee_name, status)
VALUES
(7002, 1002, 2, 'FD2026000002', 50000.00, 6.90, 180, DATE '2026-03-02', DATE '2026-08-29', 'Y', 'Rahul Sharma',
(7003, 1003, 3, 'FD2026000003', 100000.00, 7.25, 365, DATE '2026-03-03', DATE '2027-03-03', 'Y', NULL,
(7004, 1004, 4, 'FD2026000004', 150000.00, 7.40, 730, DATE '2026-03-04', DATE '2028-03-03', 'N', 'Suresh Iyer');
```

Expected: `INSERT 0 3`. One statement, one round trip, one transaction. Loading a thousand rows this way instead of a thousand separate statements is typically ten to fifty times faster.

Step 3 - Load the rest from the script

The remaining sixteen rows are in `00_Programme_Overview/sql/03_module02_fixed_deposit_data.sql`. It contains all twenty, so remove the four you already inserted first:

```
DELETE FROM fixed_deposit;
\i '00_Programme_Overview/sql/03_module02_fixed_deposit_data.sql'
```

Step 4 - Verify

```
SELECT COUNT(*)           AS deposits,
       SUM(principal_amount) AS total_book,
       MIN(booked_date)    AS first_booking,
       MAX(maturity_date)  AS last_maturity
FROM   fixed_deposit;
```

```
deposits | total_book | first_booking | last_maturity
-----+-----+-----+-----
      20 | 13650000.00 | 2026-03-01   | 2031-03-17
```

Step 5 - INSERT ... SELECT

Retail wants a separate list of high-value deposits for the wealth team.

```
CREATE TABLE fd_high_value (  
    deposit_id      INTEGER      NOT NULL,  
    customer_id     INTEGER      NOT NULL,  
    principal_amount NUMERIC(15,2) NOT NULL,  
    flagged_on      DATE          DEFAULT CURRENT_DATE NOT NULL,  
    CONSTRAINT pk_fd_high_value PRIMARY KEY (deposit_id)  
);  
  
INSERT INTO fd_high_value (deposit_id, customer_id, principal_amount)  
SELECT deposit_id, customer_id, principal_amount  
FROM   fixed_deposit  
WHERE  principal_amount >= 1000000;  
  
SELECT * FROM fd_high_value ORDER BY principal_amount DESC;
```

Expected: 6 rows, the deposits of 1,000,000 and above.

What just happened

Three loading techniques: single row, multi-row, and query-driven. The third is the one that matters most later - `INSERT ... SELECT` is how data moves between tables during ETL in Module 10 and migration in Module 11.

Lab 2.6 - The maturity run: safe UPDATE and DELETE

Objective: build the habit that stops you destroying a production table.

Step 1 - SELECT before you UPDATE

The nightly job marks matured deposits. Write the `SELECT` first.

```
SELECT deposit_id, receipt_number, maturity_date, status
FROM   fixed_deposit
WHERE  maturity_date <= DATE '2026-06-30'
      AND status = 'ACTIVE';
```

Count the rows. Whatever that count is, the `UPDATE` must change exactly that many.

```
UPDATE fixed_deposit
SET    status = 'MATURED'
WHERE  maturity_date <= DATE '2026-06-30'
      AND status = 'ACTIVE';
```

Expected: `UPDATE 4`, matching the four rows your `SELECT` returned - FD2026000001, FD2026000007, FD2026000013 and FD2026000019. **If the two numbers differ, stop and investigate.**

Step 2 - See what a missing WHERE would have done

```
BEGIN;
UPDATE fixed_deposit SET status = 'MATURED';
SELECT status, COUNT(*) FROM fixed_deposit GROUP BY status;
ROLLBACK;

SELECT status, COUNT(*) FROM fixed_deposit GROUP BY status;
```

Inside the transaction every deposit in the bank was matured. After the rollback, reality is restored. **In a real session with autocommit on, there is no rollback.** Wrapping risky DML in an explicit transaction is a professional habit worth forming today.

Step 3 - Relative versus absolute update

A customer adds 50,000 to an existing deposit at booking time.

```
SELECT principal_amount FROM fixed_deposit WHERE deposit_id = 7005;

UPDATE fixed_deposit
SET    principal_amount = principal_amount + 50000
WHERE  deposit_id = 7005;

SELECT principal_amount FROM fixed_deposit WHERE deposit_id = 7005;
```

Reverse it:

```
UPDATE fixed_deposit
SET    principal_amount = principal_amount - 50000
WHERE  deposit_id = 7005;
```

Step 4 - Update several columns

The customer breaks deposit 7006 early, so auto-renew must also be switched off.

```
UPDATE fixed_deposit
SET    status = 'BROKEN',
      auto_renew = 'N'
WHERE  deposit_id = 7006;

SELECT deposit_id, status, auto_renew FROM fixed_deposit WHERE deposit_id = 7006;
```

Step 5 - Soft close instead of DELETE

```
-- The banking way
UPDATE fixed_deposit SET status = 'CLOSED' WHERE deposit_id = 7006;

-- Rather than
-- DELETE FROM fixed_deposit WHERE deposit_id = 7006;
```

The row survives, so next year's report can still explain last year's interest payout, and the auditor can follow the trail. Restore it:

```
UPDATE fixed_deposit SET status = 'BROKEN' WHERE deposit_id = 7006;
```

What just happened

You practised the sequence that separates a careful engineer from a costly one: select, count, update, verify the count. Every experienced data engineer has a story about the day they forgot.

Lab 2.7 - Filtering, NULL and sorting

Objective: answer real questions about the Smart Saver book.

Step 1 - Simple filters

```
-- Deposits of 100,000 or more, largest first
SELECT receipt_number, principal_amount, tenure_days, maturity_date
FROM   fixed_deposit
WHERE  principal_amount >= 100000
ORDER  BY principal_amount DESC;

-- Deposits booked in the first week of March
SELECT receipt_number, booked_date
FROM   fixed_deposit
WHERE  booked_date >= DATE '2026-03-01'
      AND booked_date < DATE '2026-03-08'
ORDER  BY booked_date;

-- Deposits at the three metro branches
SELECT receipt_number, branch_id, principal_amount
FROM   fixed_deposit
WHERE  branch_id IN (1, 3, 5)
ORDER  BY branch_id, principal_amount DESC;
```

Step 2 - The BETWEEN trap, demonstrated

```
-- On a DATE column, BETWEEN is fine
SELECT COUNT(*) FROM fixed_deposit
WHERE  booked_date BETWEEN DATE '2026-03-01' AND DATE '2026-03-10';

-- On a TIMESTAMP column, it is not
SELECT COUNT(*) AS wrong FROM account_txn
WHERE  txn_date BETWEEN DATE '2026-01-01' AND DATE '2026-03-31';

SELECT COUNT(*) AS right FROM account_txn
WHERE  txn_date >= DATE '2026-01-01' AND txn_date < DATE '2026-04-01';
```

The second count is larger. The difference is every transaction that happened after midnight on 31 March - a full day of the bank's business, quietly missing from the first report.

Step 3 - Operator precedence

```
SELECT COUNT(*) AS without_brackets FROM fixed_deposit
WHERE  branch_id = 1 OR branch_id = 2 AND principal_amount > 500000;

SELECT COUNT(*) AS with_brackets FROM fixed_deposit
WHERE  (branch_id = 1 OR branch_id = 2) AND principal_amount > 500000;
```

The counts differ. **AND** binds tighter than **OR**, so the first query returns every branch 1 deposit regardless of size, plus large branch 2 deposits.

Step 4 - NULL behaviour

```
-- Wrong: returns nothing, ever
SELECT COUNT(*) FROM fixed_deposit WHERE nominee_name = NULL;

-- Right
SELECT COUNT(*) FROM fixed_deposit WHERE nominee_name IS NULL;
```

Now the counting puzzle:

```
SELECT COUNT(*) AS all_rows,
       (SELECT COUNT(*) FROM fixed_deposit WHERE nominee_name = 'Priya Verma') AS is_priya,
       (SELECT COUNT(*) FROM fixed_deposit WHERE nominee_name <> 'Priya Verma') AS not_priya
FROM   fixed_deposit;
```

`is_priya + not_priya` is **less than** `all_rows`. The rows with a NULL nominee satisfy neither test, because a comparison with NULL is neither true nor false.

To include them, be explicit:

```
SELECT COUNT(*) FROM fixed_deposit
WHERE  nominee_name <> 'Priya Verma' OR nominee_name IS NULL;
```

Step 5 - COALESCE and CASE

```
SELECT receipt_number,
       COALESCE(nominee_name, 'NOT NOMINATED') AS nominee,
       CASE WHEN principal_amount >= 1000000 THEN 'HIGH VALUE'
            WHEN principal_amount >= 100000 THEN 'STANDARD'
            ELSE 'SMALL'
       END AS segment
FROM   fixed_deposit
ORDER  BY principal_amount DESC;
```

Step 6 - Sorting with NULLs

```
SELECT receipt_number, nominee_name
FROM   fixed_deposit
ORDER  BY nominee_name;           -- NULLs last on PostgreSQL

SELECT receipt_number, nominee_name
FROM   fixed_deposit
ORDER  BY nominee_name NULLS FIRST; -- the follow-up list
```

Where NULLs sort by default differs between engines. Be explicit whenever it matters.

Step 7 - The deliverable

The call centre's list. This is what ticket ICICI-4482 asked for.

```

SELECT  fd.receipt_number,
        c.first_name || ' ' || c.last_name      AS customer_name,
        c.phone,
        fd.principal_amount,
        fd.maturity_date,
        fd.auto_renew,
        COALESCE(fd.nominee_name, 'NOT NOMINATED') AS nominee
FROM    fixed_deposit fd
JOIN    customer c ON c.customer_id = fd.customer_id
WHERE   fd.status = 'ACTIVE'
AND     fd.maturity_date >= DATE '2026-05-25'
AND     fd.maturity_date < DATE '2026-06-20'
ORDER   BY fd.maturity_date, fd.principal_amount DESC;

```

receipt_number	customer_name	phone	principal_amount	maturity_date	auto_renew	nominee
FD2026000001	Aarav Sharma	9810000000	25000.00	2026-05-30	N	Priya Verma
FD2026000007	Vikram Patel	9810000822	75000.00	2026-06-05	N	Anita Nair
FD2026000013	Grace Mehta	9810002329	100000.00	2026-06-11	N	Priya Verma
FD2026000019	Pooja Rao	9810003699	150000.00	2026-06-17	N	Anita Nair

Four deposits mature in that window, worth 2,375,000 between them, and none of them auto-renews - so the call centre has four conversations to have. The date range is fixed rather than `CURRENT_DATE` so that everyone in the room gets the same rows.

What just happened

You answered seven real questions and met the three filtering mistakes that cause the most production defects: `BETWEEN` on a timestamp, unbracketed `AND/OR`, and comparing to `NULL` with `=`.

Lab 2.8 - DELETE, TRUNCATE and DROP compared

Objective: never confuse the three again.

```
-- Make a copy to experiment on
CREATE TABLE fd_scratch AS SELECT * FROM fixed_deposit;
SELECT COUNT(*) FROM fd_scratch;    -- 20
```

DELETE - selective and reversible

```
BEGIN;
DELETE FROM fd_scratch WHERE principal_amount < 100000;
SELECT COUNT(*) FROM fd_scratch;    -- fewer
ROLLBACK;
SELECT COUNT(*) FROM fd_scratch;    -- 20 again
```

TRUNCATE - all rows, table remains

```
BEGIN;
TRUNCATE TABLE fd_scratch;
SELECT COUNT(*) FROM fd_scratch;    -- 0
ROLLBACK;
SELECT COUNT(*) FROM fd_scratch;    -- 20 again, because PostgreSQL makes TRUNCATE transactional
```

On Oracle and MySQL this rollback does not work. `TRUNCATE` is DDL there, it commits implicitly, and the rows are gone. Never test that assumption on a production table.

DROP - the table itself

```
DROP TABLE fd_scratch;
SELECT COUNT(*) FROM fd_scratch;
```

```
ERROR:  relation "fd_scratch" does not exist
```

The one you cannot do

```
DROP TABLE customer;
```

```
ERROR:  cannot drop table customer because other objects depend on it
DETAIL:  constraint fk_account_customer_id on table account depends on table customer
```

Referential integrity protects the schema itself, not only the rows.

What just happened

Statement	Rows	Table	Rollback	Speed
DELETE	Chosen	Stays	Yes	Slow on big tables
TRUNCATE	All	Stays	PostgreSQL and SQL Server only	Instant
DROP	All	Gone	Rarely	Instant

End-of-module checklist

	Item
[]	I can explain, with output, why money is <code>NUMERIC</code> and not <code>FLOAT</code>
[]	<code>fixed_deposit</code> exists with 11 columns and 10 named constraints
[]	I have seen the error message from all six constraint types
[]	I have added a column, widened a type and replaced a check constraint
[]	The 20 pilot bookings are loaded and total 13,650,000
[]	I write <code>SELECT</code> with the intended <code>WHERE</code> before any <code>UPDATE</code> or <code>DELETE</code>
[]	I can explain why <code>nominee_name = NULL</code> returns nothing
[]	I can state the difference between <code>DELETE</code> , <code>TRUNCATE</code> and <code>DROP</code>

Common problems and fixes

Symptom	Cause	Fix
<code>relation "fixed_deposit" already exists</code>	Table created twice	<code>DROP TABLE fixed_deposit;</code> and re-create
<code>violates foreign key constraint fk_fd_customer</code>	<code>customer_id</code> not in <code>customer</code>	Use a real id from <code>SELECT customer_id FROM customer</code>
<code>column contains null values on ALTER</code>	Adding <code>NOT NULL</code> to a populated table	Add <code>DEFAULT</code> in the same statement
<code>UPDATE 20</code> when you expected 3	Missing or wrong <code>WHERE</code>	Roll back; run the <code>SELECT</code> version first
Query returns 0 rows unexpectedly	Compared to <code>NULL</code> with <code>=</code>	Use <code>IS NULL</code>
Quarterly report short by a day	<code>BETWEEN</code> on a timestamp	Use <code>>= start AND < next_start</code>
<code>operator does not exist: character varying + integer</code>	Used <code>+</code> to concatenate	Use the double-pipe operator in PostgreSQL